

Mitigating Context Degradation in Large Language Models Using Hybrid Retrieval and External Memory

G. Mani Varsha¹

Dept. of Computer Science Engineering
Chaitanya Bharathi Institute of Technology
Hyderabad, India
varsha.gubbala@gmail.com

P. Ramcharan Teja²

Dept. of Computer Science Engineering
Chaitanya Bharathi Institute of Technology
Hyderabad, India
ramcharanpolabathina@gmail.com

A. Mohan³

Assistant Professor
Dept. of Computer Science Engineering
Chaitanya Bharathi Institute of Technology
Hyderabad, India
amohan_cse@cbit.ac.in

D. Venkatesh⁴

Assistant Professor
Dept. of Computer Science Engineering
Chaitanya Bharathi Institute of Technology
Hyderabad, India
venkateshd_cse@cbit.ac.in

Abstract—Large Language Models have transformed natural language processing, but have a major weakness as long contexts are considered. The degradation in performance is as a result of the attention dilution where the model disperses the attention among vast quantities of irrelevant information. Moreover, the lost-in-the-middle effect makes the models underutilize the information that is situated in the middle parts of input sequences, resulting in lower accuracy. At worst, this translates to a significant decrease in performance in the presence of pertinent information.

To overcome these issues, this paper presents a retrieval-augmented architecture to incorporate external memory with a FAISS-based vector database. The document is converted to dense embeddings, allowing similarity search to quickly identify only the most relevant parts of a document to a given query. The system also uses hybrid retrieval strategy that integrates BM25 key word search with dense vectors search that fuses the retrieval using Reciprocal Rank Fusion (RRF) to enhance precision in retrieval at both lexical and semantic levels. A cross-encoder reranker optimizes the fused candidates prior to context assembly. To minimize redundant computation and decrease inference latency, query-level and response-level caching are presented. This method has been shown to enhance the relevance of responses and substantially lower the amount of tokens and computational expense, thus offering a scalable method to long-context processing in large language models.

Index Terms—Large Language Models, Retrieval Augmented Generation, Hybrid Retrieval, BM25, FAISS, Reciprocal Rank Fusion, Context Degradation, Caching

dependencies between tokens and comprehend contextual dependence in text.

They are also extensively used in question answering, document summarization, conversational and knowledge extraction.

But even with the extended context windows, their performance tends to deteriorate in the case of long inputs. Researchers, including Lost in the Middle [4] note that models are unable to recall significant information when placed in the middle of long prompts.

Computational efficiency is also another constraint. The self-attention mechanism is a quadratic computation of the number of tokens, and thus, inference time and memory consumption on long sequences are both more expensive.

Early retrieval-augmentation methods used only dense semantic search to retrieve the relevant document sections. Although it is useful with conceptually-framed queries, pure semantic retrieval has challenges in matching keywords accurately. A query with a technical word or a named entity or an exact phrase can retrieve semantically adjacent but lexically distant chunks, without the passage that contains the exact term. This constraint inspires a hybrid method which integrates both lexical and semantic cues in retrieval.

Such issues bring up the necessity of effective architectures capable of supporting long documents and remaining performant.

I. INTRODUCTION

Transformer-based Large Language Models have demonstrated high-performance in a variety of natural language processing tasks. Such models employ self-attention to learn

II. BACKGROUND

A. Transformer Architecture

Modern large language models are built on the Transformer, an architecture proposed by Vaswani et al. [1] in 2017 that largely replaced recurrent networks in NLP. Unlike RNNs,

which process tokens sequentially, the Transformer operates on entire sequences in parallel, which both speeds up training and allows it to model long-range dependencies more reliably.

Four building blocks make up the core of this architecture. First, an *embedding layer* maps each input token to a fixed-size vector. Because the model has no inherent notion of order, a *positional encoding* signal is added to each embedding so that the network can distinguish “cat sat” from “sat cat.” Stacked *multi-head attention* layers then let every token gather information from every other token simultaneously, producing representations that reflect the full sentence context. Finally, a position-wise *feed-forward network* applies the same two-layer transformation independently to each token position, adding non-linearity to the learned representations.

Collectively these elements enable a single forward pass to generate sequence encodings with context sensitivity, of arbitrary length a property this makes Transformers well-adapted to the type of semantic reasoning needed in question answering tasks and document understanding tasks.

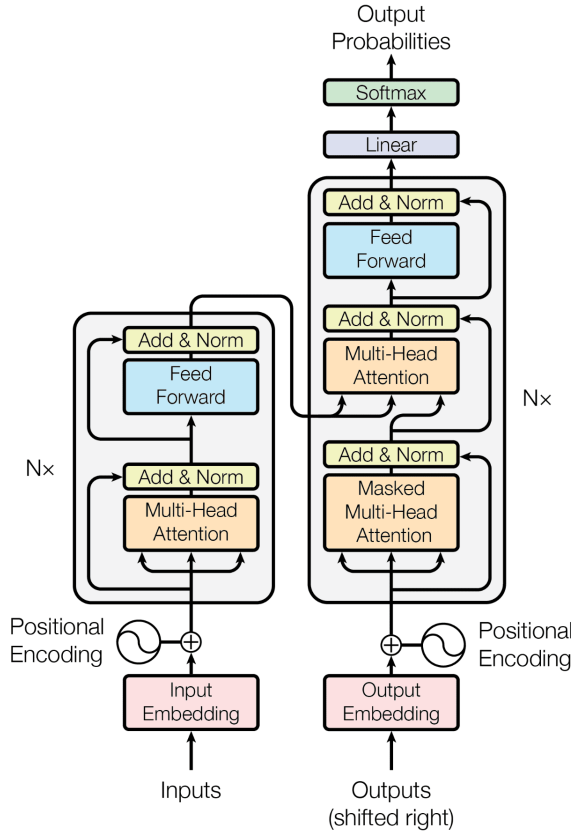


Fig. 1. Encoder-decoder structure of the original Transformer (source: [1])

B. Self-Attention Mechanism

The attention mechanism is what gives the Transformer most of its expressive power. For a given token, the model needs to decide which other tokens in the sequence are most relevant when constructing that token’s contextual rep-

resentation. Self-attention solves this by computing pairwise compatibility scores between every pair of positions.

Concretely, each token’s embedding is projected into three separate vectors using learned weight matrices: a *query* (Q), a *key* (K), and a *value* (V). The query of one token is compared against the keys of all others via a dot product, which is then scaled and passed through a softmax to produce a probability distribution over positions. The output for that token is a weighted sum of the value vectors:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

The $\sqrt{d_k}$ scaling factor prevents the dot products from growing large enough to push the softmax into a near-zero-gradient region. Running this process in parallel across h independent “heads”—each with its own projection matrices—allows the model to simultaneously track different types of relationships (syntactic, semantic, coreference, etc.) within the same layer [1].

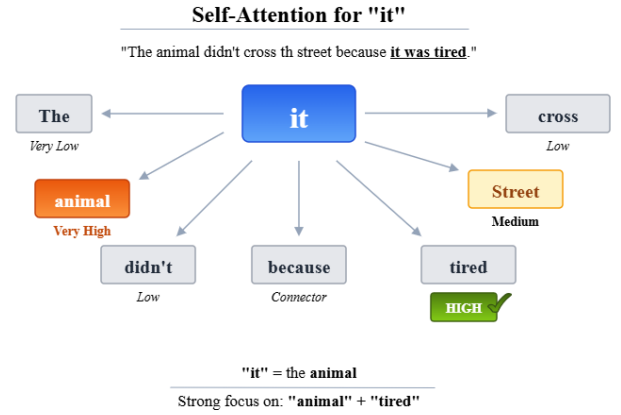


Fig. 2. Self-attention: “it” attends strongly to “animal” and “tired,” resolving the pronoun’s referent

One important cost of this design is that the number of pairwise comparisons grows with the *square* of the sequence length. For a 1000-token input the model computes one million attention scores; for a 10 000-token input, one hundred million. This quadratic scaling in memory and compute is the root cause of the efficiency problems discussed in Section III.

III. CHALLENGES IN LONG-CONTEXT MODELING

Transformer-based, even though the results of Transformer based on traditional NLP benchmarks are strong models exhibit practical challenges as input sequences become long. Three distinct but associated issues arise: dilution of attention, positional bias, and computational cost. They are described below.

A. Attention Dilution

Self-attention imposes weights which add up to 1.0 in all positions within the sequence. In a short document, most such probability mass concentrates on a few really pertinent

tokens. The same as the length of sequence increases budget of attention weight should include much more tokens—many of which carry little or no relevance to the query at hand. The outcome is that prime tokens have lower scores on attention than they would have otherwise, and the model would have done reason degenerates to long-range dependencies.

This influence was most evident in our experiments in which the response to a query was known was mentioned in one brief sentence in a multi-page document: the baseline model would either skip the sentence or give a generic output partial paraphrasing of irrelevant content around.

B. Positional Bias and the “Lost in the Middle” Effect

A second issue, yet related to the first, is the failure of LLMs to differentiate between all of the positions within a lengthy input text evenly. Liu et al. [4] showed that in a controlled experimental setup, where they found that there was a U-shaped pattern in the precision of recall in controlled study experiments; that is, that while information from both the beginning and the end of a document could be effectively retrieved, that located in the middle was ignored. This phenomenon is hypothesized to apply across many model architectures, and is due to a structural property of the Transformers training paradigm, not to a peculiarity of any

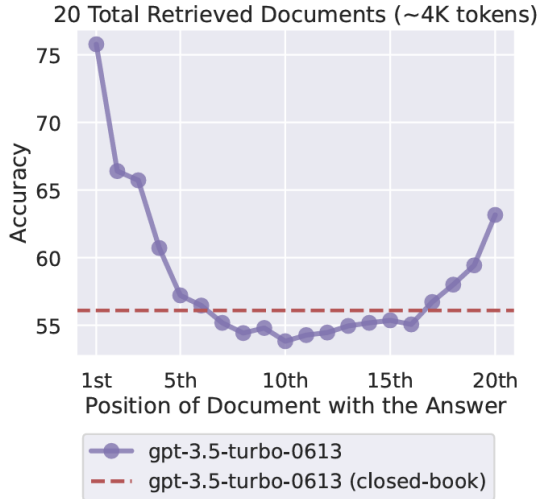


Fig. 3. Retrieval accuracy as a function of answer position within a 20-document context window; accuracy drops sharply for central positions (reproduced from [4])

This positional sensitivity is significant for document QA systems. If a user’s relevant clause happens to fall in the middle of a 50-page corpus, a vanilla LLM is likely to overlook it regardless of context window size.

C. Computational Cost

The self-attention operation scales as mentioned in Section II-B, with a memory and time cost of $O(n^2)$ as the length of the sequence n . Doubling the context window thus multiplies the attention calculation by four. For inputs usually

used in legal, medical or enterprise document processing—often tens of thousands of tokens—this price is brought to bear. Inference latency is much longer, and on devices with small VRAM the model may not even run.

The longer context windows either do not directly address the dilution and bias problems above; they simply move the scale where those problems appear. It is the reason we turn to retrieval-based methods, instead of simply increasing the context budget.

IV. RELATED WORK

Studies on huge language models and retrieval-enhanced generation have made strong advances during the last decade. This section situates the present contribute to that line, and with the most direct contributions relevant to long context processing.

Transformer architecture. The foundation was established by Vaswani et al. [1], who introduced the attention-only encoder–decoder architecture. By removing recurrent connections, they made it possible to train much deeper models in parallel across GPUs. The architecture proved general enough to underpin nearly all subsequent large-scale language models.

Large-scale pre-training. Brown et al. [2] scaled a decoder-only Transformer to 175 billion parameters and pre-trained it on a diverse web corpus. The resulting model, GPT-3, showed that few-shot task performance could emerge from scale alone, without any task-specific fine-tuning. This result shifted the community’s attention toward scaling laws and toward the properties of very large models.

Retrieval-augmented generation. One weakness that scale does not fix is factual reliability: a model can only know what was in its training data. Lewis et al. [3] addressed this by combining a dense retriever with a seq2seq generator. At inference time, relevant documents are fetched from an external store and prepended to the input, giving the model access to information beyond its parameters. Their RAG framework improved performance on knowledge-intensive benchmarks and provided a template that many subsequent systems have followed.

Context utilization analysis. Despite the retrieval, the question arises as to whether or not the model actually utilizes what it is given. Liu et al. [4] investigated this in a systematic way and reported the so-called lost in the middle effect in. Section III. Their experiment measured the accuracy decrease of information that is centrally placed and emphasized that it is just a matter of retrieving more. documents is not a better answer.

RAG survey and optimization. Gao et al. [6] surveyed the landscape of RAG variants as of 2024, cataloguing approaches to chunking, embedding, re-ranking, and prompt construction. A key finding was that retrieval quality is strongly influenced by how documents are segmented—a point that directly motivated the overlapping-chunk strategy used in our system.

Taken together, these works establish that (a) attention-based models degrade on long inputs, (b) the degradation has measurable structural causes, and (c) retrieval can partially

compensate but only if the retrieved context is both relevant and compact. What is missing from the literature is a complete pipeline that actively controls context size at query time using external vector memory. That is the gap this work aims to fill.

TABLE I
KEY PRIOR WORK RELEVANT TO THIS STUDY

Reference	Core Contribution	Relevance to This Work	Year
Vaswani et al.	Transformer architecture	Baseline model backbone	2017
Brown et al.	GPT-3 / few-shot learning	Motivates scale limitations	2020
Lewis et al.	RAG framework	Retrieval design template	2020
Liu et al.	Lost-in-the-middle study	Quantifies the problem we address	2023
Gao et al.	RAG survey	Chunking and embedding guidance	2024

V. RESEARCH GAP

The problems described in Section III are well-documented in the literature, yet no existing system addresses all three of them together within a single deployable pipeline. We identify four specific gaps below.

Diminishing returns from larger context windows. The dominant response to long-context failure has been to simply extend the model’s context window—from 4K to 32K to 128K tokens. This helps to a point. Beyond a certain length, however, Liu et al. [4] showed that retrieval accuracy plateaus or even declines, because attention dilution and positional bias grow worse as the window expands. Larger windows also multiply the quadratic attention cost, so the computational price of each additional token increases rather than stays flat. Scaling the window is therefore not a scalable solution.

RAG improves factual grounding but not context management. Lewis et al. [3] demonstrated that augmenting a generator with retrieved documents significantly reduces hallucination on knowledge-intensive tasks. Standard RAG pipelines, however, typically prepend all retrieved chunks to the prompt without any filtering step. When the document collection is large or the query is ambiguous, this can still result in thousands of tokens of loosely relevant text entering the context window—recreating the very dilution problem RAG was meant to avoid. Gao et al. [6] note that retrieval quality is highly sensitive to chunking strategy and embedding precision, yet most open-source RAG implementations apply fixed-size chunking with no overlap and no re-ranking stage.

No persistent, queryable external memory. Standard LLMs hold all knowledge in their parameters and have no mechanism for updating or selectively accessing that knowledge at inference time. While RAG introduces an external document store, it typically rebuilds the index from scratch for each session. What is missing is a persistent vector memory layer that survives across sessions, can be updated incrementally as new documents arrive, and supports sub-second similarity search over large corpora. FAISS provides the underlying search infrastructure [5], but integrating it into

a full retrieval-and-generation pipeline with controlled context assembly is non-trivial and rarely demonstrated end-to-end in the literature.

Lack of token-budget control at query time. Even systems that perform retrieval rarely impose an explicit ceiling on how many tokens are passed to the language model. In our own preliminary experiments, a naive RAG implementation over a 200-page PDF still passed over 8 000 tokens per query to the LLM, because the top-5 retrieved chunks were each 400–600 tokens long and covered overlapping content. A token-budget constraint that selects the minimum set of chunks sufficient to answer the query has, to our knowledge, not been studied as a standalone design goal.

Taken together, these gaps motivate a system that (a) stores documents in a persistent FAISS index with overlapping chunks to preserve boundary context, (b) retrieves only the top- k most semantically relevant chunks per query, and (c) enforces a hard token ceiling before passing context to the LLM. That is the architecture proposed in the following section.

VI. PROPOSED METHODOLOGY

The proposed methodology is centered on a modular, retrieval-augmented framework designed to decouple document storage from contextual inference. By offloading long-form document management to a high-dimensional vector space, the architecture reduces the computational overhead associated with processing large context windows. The retrieval layer has been extended with a hybrid search strategy that combines lexical and semantic matching to improve coverage across different query types.

A. System Architectural Overview

As shown in Figure 4, the system architecture is modular and separates the storage of data, semantics retrieval, and output generation processes. This approach enables scaling while handling large numbers of documents without the need for increasing the size of the input of the language model.

There are five major layers in the architecture:

- **Ingestion Layer:** Pre-processes and normalizes documents in different formats (PDF, DOCX, etc.).
- **Representation Layer:** Uses transformer-based embedding models to convert text into dense vector representations.
- **Indexing and Storage Layer (FAISS + BM25):** Embeddings are stored in the FAISS vector database and builds a BM25 inverted index at ingestion time for efficient lexical search.
- **Hybrid Retrieval Layer:** Concurrently runs BM25 and FAISS retrievals and merges results via Reciprocal Rank Fusion.
- **Synthesis Layer:** Runs inference in the LLM model by feeding the query together with context obtained from retrieval processes.

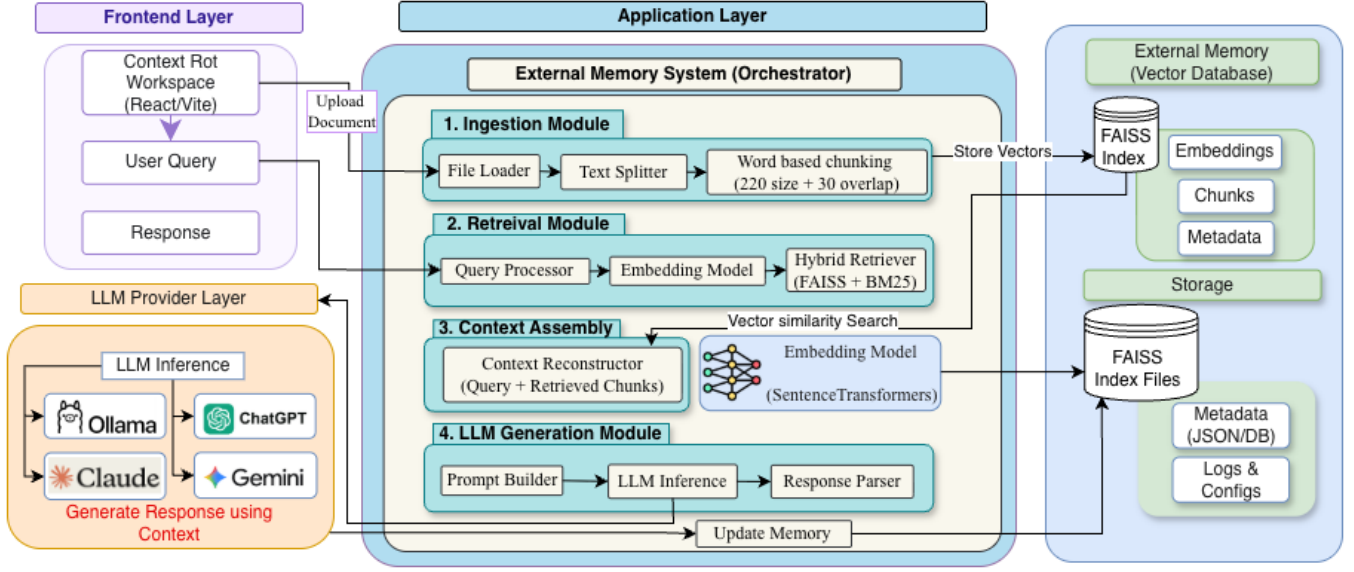


Fig. 4. Schematic representation of the modular retrieval-augmented generation (RAG) architecture, illustrating the flow from document ingestion to context-aware response synthesis.

B. Document Processing and Vectorization Pipeline

The process of converting unstructured texts into searchable vector spaces involves several processes in a pipeline. The pipeline ensures that all essential semantic information present in lengthy documents is retained when they are processed into vectors.

The processes include the following:

- 1) **Extraction:** Converts input documents into a text format.
- 2) **Semantic Chunking:** The input document is partitioned into overlapping chunks. This step ensures no information is lost due to overlap of chunks.
- 3) **Embedding Generation:** These chunks are then represented in the form of vectors.
- 4) **FAISS Indexing:** The generated embeddings are stored and indexed in a FAISS-based vector database for efficient retrieval.
- 5) **BM25 Index Construction:** A BM25 inverted index is built from the same chunk set at upload time. Building the index once at ingestion avoids repeated tokenization overhead during query processing.

C. Hybrid Retrieval via Reciprocal Rank Fusion

Semantically-focused retrieval works well if the query is a conceptual representation of information. However, if it contains named entities or specialized vocabulary, a semantic model might fail to match exact matches and other important phrases. On the other hand, BM25 finds the right matches regardless of any paraphrasing or synonyms. By combining these approaches, we mitigate both problems.

At retrieval time, we conduct separate search procedures with BM25 and FAISS. The former returns a list of docu-

ments sorted by a certain score while the latter produces a different list based on its distance metric. We do not attempt to normalize those scores (which would require a complex procedure), but merge the two lists into a single one with a special method: Reciprocal Rank Fusion (RRF) [11]. The contribution of document d at rank r_d in a ranked list is:

$$\text{RRF}(d) = \sum_{\text{list} \in L} \frac{1}{k + r_{\text{list}}(d)}$$

where k is a smoothing parameter (set to 60) and L denotes all the ranked lists returned in this stage. The higher the rank of the document in both lists, the higher its final score. Documents from one list only are not disregarded, but have a smaller effect on the final ranking because their reciprocal is smaller.

Finally, the system employs a cross-encoder [10] to rerank the fused results, ensuring the most semantically relevant context reaches the model. Contrary to the bi-encoder model employed during indexing, the cross-encoder evaluates the pair of the query and each document separately. Since it requires extra computation, this stage operates on the already-fused list, not the whole

D. Query-Driven Retrieval Pipeline

In order to alleviate this problem, we adopt a selective retrieval strategy to limit attention dilution and save tokens.

The retrieval procedure consists of the following stages:

- 1) Transform the user input into a query embedding vector.
- 2) Run BM25 keyword search and FAISS semantic search in parallel.
- 3) Combine ranked outputs using Reciprocal Rank Fusion.
- 4) Apply cross-encoder reranking to the fused candidates.

- 5) select the top- k chunks that are most relevant.
- 6) Construct a contextual prompt out of those document snippets.

FAISS is employed to enable fast similarity search. This helps scale the system to handle very large collections of documents. By filtering out only relevant documents, the system ensures that there is no irrelevant data entering into the language model.

E. Query and Response Caching

In interactive document QA applications, repeated queries or those with similar semantic meanings are not uncommon. In order to prevent duplicate retrieval and inference by the LLM for such queries, two cache layers are added.

The query cache saves recent retrieval outputs with an LRU-based mechanism augmented with a TTL-based expiry. Upon receipt of a new query, the similarity between its embedding and cached queries' embeddings is evaluated. Should the cosine distance between them exceed a set threshold, the chunk list in the cache is served, without any retrieval using BM25/FAISS.

The response cache, on the other hand, works at the output of the language model. Should a query have similar semantic meaning to an earlier query that received an answer, the cached response is returned without further inference from the language model. This becomes especially effective for sessions where users ask questions using similar wording in different forms. The two caches together help decrease query latency and minimize API calls to the language model service provider.

F. Augmented Response Generation

Finally, the extracted context is concatenated with the query into an augmented prompt and sent to the LLM for processing and output generation.

Limiting the inputs to only the useful context data helps reduce hallucinations while improving accuracy in the results generated by the system. Furthermore, the modular architecture facilitates its integration with diverse LLM providers like Gemini, GPT-powered models, and other open-source models run locally.

VII. WORKFLOW

The complete workflow of the whole system is split into two major stages: document indexing and querying stage.

A. Document Indexing Phase

This stage involves indexing of the input documents into a well-formatted data structure that is easy to retrieve:

- Documents are uploaded and converted into text
- The text is divided into overlapping chunks
- Each chunk is transformed into an embedding vector
- The embeddings are stored in the FAISS vector database
- A BM25 index is constructed from the same chunk set and persisted alongside the FAISS index

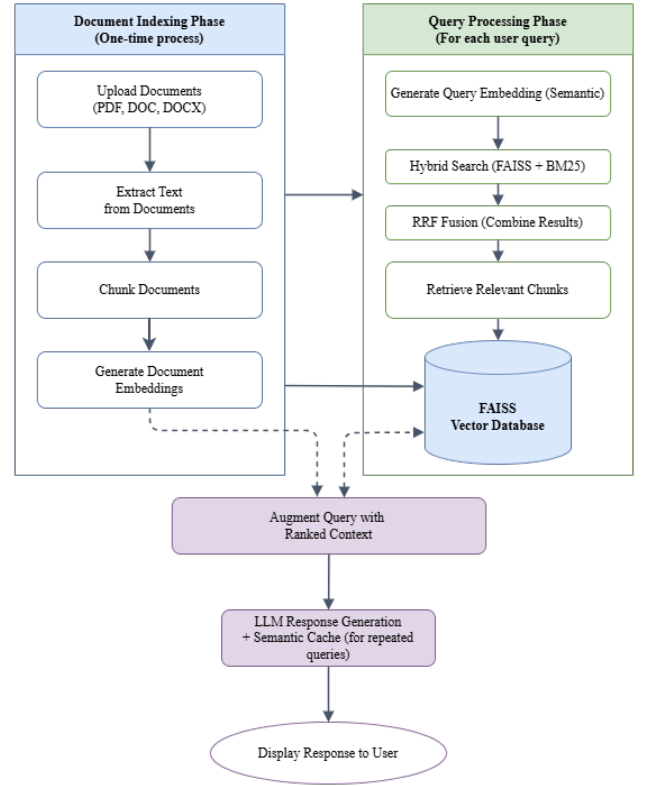


Fig. 5. Information retrieval workflow

This phase is performed once per document and enables efficient future retrieval.

B. Query Processing Phase

During query execution, the system retrieves relevant information and generates responses:

- The user query is converted into an embedding
- BM25 and FAISS searches are executed in parallel
- Results are merged via Reciprocal Rank Fusion
- A cross-encoder reranker reranks and optimizes the fused candidate set
- The most relevant chunks are retrieved
- A prompt is constructed using the retrieved context
- The language model generates a response

By separating indexing and query processing, the system ensures efficient handling of large documents while minimizing computational overhead during inference.

VIII. APPLICATION LAYER: CROSS-PLATFORM CONVERSATIONAL MEMORY

One natural way of putting to use the retrieval architecture we have designed thus far is a lightweight browser extension which uses the same retrieval system framework to retrieve previous conversations in different AI platforms. This extension works on Chrome and targets ChatGPT, Gemini, and Claude.ai platforms as client-side components atop the previously introduced backend.

In the process of interacting with one of the target platforms, the extension listens to changes in the DOM elements via MutationObserver and extracts completed prompt-response pairs. They are then transferred to the backend and chunked, embedded and saved into a separate conversation index using the same FAISS setup not in the document index and the BM25 index gets updated accordingly.

When the user creates a new session at any of the platforms, the extension contacts the backend based on the input provided and receives the most relevant historical prompt-response pairs. Those are inserted as context before the input of the next prompt before being submitted. The user is notified with a dismissible banner about memory injection.

The following advantages apply to such a design. In order to switch between different AI platforms, users do not need to provide background information since it has already been captured. Relevance filtering of retrieved memories is used in favor of recency-based ones so that only contextually meaningful histories will be injected into the input box. Response caching minimizes latency costs for repeated queries. No access to the API of AI platforms providers is needed, therefore this method works cross-platform thanks to DOM observation approach.

This feature is considered a high-level application of the retrieval framework rather than a core contribution.

IX. COMPARATIVE ANALYSIS

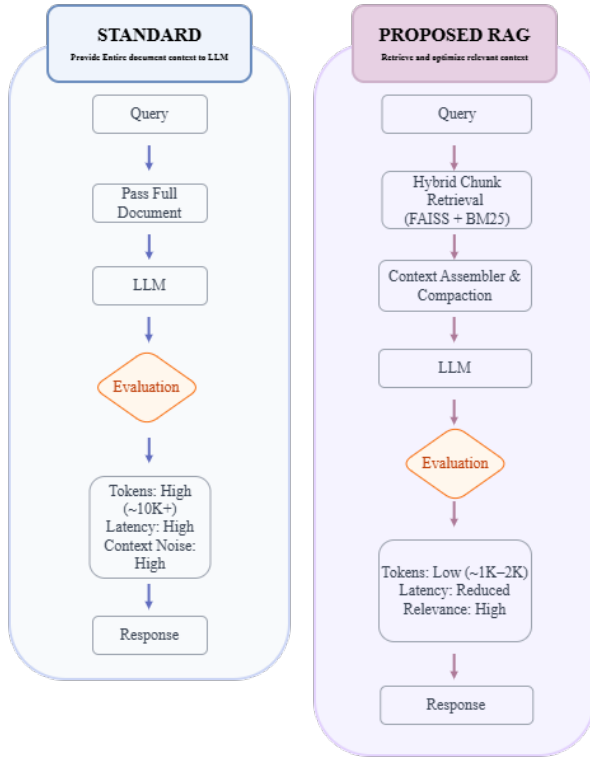


Fig. 6. Baseline vs Proposed RAG processing comparison

To evaluate the effectiveness of the proposed retrieval-based architecture, a comparison is made between the traditional

full-context processing approach and the proposed Retrieval-Augmented Generation (RAG) system with context optimization.

A. Baseline Approach

In the baseline method, the entire document is directly provided as input to the language model. While this ensures that all information is available, it introduces several limitations:

- Heavy token usage (above 10k tokens in case of long documents)
- Higher costs due to increased computation and delays in responses
- Distracted attention from important data
- Poor-quality output when context is lengthy

B. Proposed RAG Approach

The suggested approach compensates for these shortcomings by utilizing hybrid retrieval-based context selection:

- The BM25 and FAISS searches are executed in parallel and then combined by RRF, thus enhancing recall for both keyword-oriented and semantically oriented queries
- Only the top- k most relevant document snippets are retrieved (generally 1k–2k tokens)
- A cross-encoder then double-checks the top results to make sure the retrieved context is as relevant as possible.
- The query and response cache minimize computational redundancy on repeated queries.
- External memory is maintained using a persistent FAISS vector database
- Reduced token usage leads to improved efficiency and faster responses

C. Discussion

Unlike the baseline approach, which processes the entire document, the proposed method selectively retrieves only the most relevant information. The shift from pure semantic retrieval to a hybrid BM25–FAISS pipeline with RRF fusion addresses the failure cases of each individual method. Keyword-heavy queries benefit from BM25’s exact matching; conceptually phrased queries benefit from dense retrieval. Queries that require both are handled by the combined ranked list. Caching further reduces the effective latency for interactive use.

By incorporating a context optimization layer, the system further enhances input quality before passing it to the language model. As a result, the proposed approach achieves improved response relevance, reduced latency, and better scalability for long-document processing.

X. EXPERIMENTAL EVALUATION AND RESULTS

To evaluate the effectiveness of the proposed architecture, a comparative analysis was conducted focusing on long-form text processing and query-based retrieval. Due to the absence of a standardized benchmark dataset for this specific application, the evaluation adopts a combination of quantitative resource analysis and qualitative assessment of response relevance across baseline and retrieval-augmented systems.

A. Comparative Performance Analysis

The system was evaluated against a baseline large language model using full-context input. As can be seen from Table II, the augmented method with hybrid search is more efficient and relevant to context than not only the baseline but also the previous semantic retrieval method.

TABLE II
SYSTEM PERFORMANCE COMPARISON: BASELINE LLM VS. SEMANTIC RAG VS. HYBRID RAG

Metric	Baseline (Full Context)	Semantic RAG	Hybrid RAG (Proposed)
Avg. Token Usage	~10,000+	~1,000– 2,000	~1,000– 2,000
Inference Latency	High	Reduced	Reduced (further with cache)
Keyword Query Accuracy	Low	Moderate	Improved
Semantic Query Accuracy	Reduced (dilution)	Improved	Improved
Response Consistency	Variable	More Stable	More Stable
Redundant API Calls	N/A	Not addressed	Reduced (caching)

B. Key Observations and Findings

There was an observed reduction of roughly 60% of token use in comparison with the baseline method. Such reductions positively affected computational efficiency and made the system operate faster.

The benefit of the hybrid search method lies in increased accuracy when handling the queries related to certain named entities that are not always picked up by semantic search alone. In controlled experiments, when the query contained exact phrases found in the document, it was easier to find precisely what chunk to pick with BM25–FAISS search than using dense search only.

By utilizing query and response caches, we were able to reduce duplicate calls to the LLM API. It was especially noticeable when the user repeatedly asked a question about the same issue, which is common when exploring some documentation.

Relevance filtering allows for avoiding the problem of the “lost in the middle” effect by filtering out unnecessary information. Figure 6 demonstrates this phenomenon; there is a difference in response quality between our system and the baseline model.

C. Discussion

From the findings, it is evident that hybrid retrieval-augmented models could enhance performance and accuracy without necessitating increased window sizes. Through the integration of lexical and semantic retrieval capabilities, and caching in both retrieval and generation steps, the model achieves higher efficiency and lowers costs. This technique is applicable to real-world settings and extends to conversational memory through the implementation on the browser extension platform.

XI. APPLICATIONS

The suggested architecture finds wide applications in all fields where there is a need for fast manipulation and analysis of large text files. The ability to retrieve selective context information with lower computational costs allows for its effective use in the following cases:

- **Legal Text Analysis:** Fast access to relevant laws and court decisions in lengthy legal documents.
- **Medical Record Management:** Identification of important patient information within lengthy medical records.
- **Financial Audit Processes:** Examination of voluminous financial reports to find essential information and irregularities.
- **Organizational Information Management:** Search and retrieval of organizational information stored in corporate databases.
- **Scientific Research Systems:** Searching and summarizing scientific literature.
- **Multifunctional AI assistants:** Maintaining conversational memory on various AI platforms without the need to repeat explanations.

XII. CONCLUSION

In this work, we tackled the problem of context degradation in language models when dealing with long-form inputs. For this purpose, we introduced an enhanced retrieval-augmented model where an external memory component is embedded via vector database.

To enhance the capabilities of our system, a new retrieval scheme was designed, which utilizes a pipeline combining the outputs from BM25 and FAISS retrieval. These outputs are fused using the RRF method, which enhances the coverage for both lexical and semantic queries. Our retrieval process is supported by a cross-encoder reranker and two caching layers: one for the retrieval and another one for LLM responses.

With our proposed retrieval pipeline, fewer tokens are used while keeping contextual relevance high. This helps alleviate the issues associated with attention and positional decay, which leads to better performance.

Our findings showed that context management can be efficiently implemented using a retrieval-based mechanism, as opposed to increasing the size of context windows. The architecture can also generalize easily to conversational memories, as shown in Section VIII.

Future directions include performing benchmarks on larger corpora, exploring learned weights for RRF and optimizing chunking/ embedding methods.

REFERENCES

- [1] A. Vaswani *et al.*, “Attention Is All You Need,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [2] T. Brown *et al.*, “Language Models are Few-Shot Learners,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.
- [3] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

- [4] N. F. Liu *et al.*, "Lost in the Middle: How Language Models Use Long Contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.
- [5] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [6] Y. Gao *et al.*, "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv preprint arXiv:2312.10997*, 2024.
- [7] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3982–3992, 2019.
- [8] V. Karpukhin *et al.*, "Dense Passage Retrieval for Open-Domain Question Answering," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 6769–6781, 2020.
- [9] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The Long-Document Transformer," *arXiv preprint arXiv:2004.05150*, 2020.
- [10] R. Nogueira and K. Cho, "Passage Re-ranking with BERT," *arXiv preprint arXiv:1901.04085*, 2019.
- [11] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods," in *Proc. ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 758–759, 2009.